1.Caesar Cipher

```
import java.util.Scanner;

class CaesarCipher {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the text: ");
        String text = sc.nextLine();

        System.out.print("Enter the key: ");
        int key = sc.nextInt();

        String encrypted = "";
        String decrypted = "";

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            if (ch >= 'A' && ch <= 'Z')
                encrypted += (char) ((ch - 'A' + key) % 26 + 'A');
            else if (ch >= 'a' && ch <= 'z')
                encrypted += (char) ((ch - 'a' + key) % 26 + 'a');
            else
                encrypted += ch;
        }

        for (int i = 0; i < encrypted.length(); i++) {
            char ch = encrypted.charAt(i);

            if (ch >= 'A' && ch <= 'Z')
                decrypted += (char) ((ch - 'A' - key + 26) % 26 + 'A');
            else if (ch >= 'a' && ch <= 'z')
                decrypted += (char) ((ch - 'a' - key + 26) % 26 + 'a');
            else
                decrypted += ch;
        }

        System.out.println("Encrypted text: " + encrypted);
        System.out.println("Decrypted text: " + decrypted);

        sc.close();
    }
}
```

2.Playfair Cipher – Java Code

```
import java.util.Scanner;

class Playfair {
   static char[][] m = {
      {'M','O','N','A','R'},
      {'C','H','Y','B','D'},
      {'E','F','G','I','K'},
      {'L','P','Q','S','T'},
      {'U','V','W','X','Z'}
   };

   static String cipher(String s, int d) {
      String out = "";

      for (int i = 0; i < s.length(); i += 2) {
         char a = s.charAt(i), b = s.charAt(i + 1);
         int r1 = 0, c1 = 0, r2 = 0, c2 = 0;

         for (int r = 0; r < 5; r++)
            for (int c = 0; c < 5; c++) {
               if (m[r][c] == a) { r1 = r; c1 = c; }
               if (m[r][c] == b) { r2 = r; c2 = c; }
            }

         if (r1 == r2) {
            out += m[r1][(c1 + d + 5) % 5];
            out += m[r2][(c2 + d + 5) % 5];
         } else if (c1 == c2) {
            out += m[(r1 + d + 5) % 5][c1];
            out += m[(r2 + d + 5) % 5][c2];
         } else {
            out += m[r1][c2];
            out += m[r2][c1];
         }
      }
      return out;
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter plaintext: ");
        String s = sc.nextLine().toUpperCase().replace("J", "I").replace(" ", "");

        if (s.length() % 2 != 0) s += "X";

        String enc = cipher(s, 1);
        System.out.println("Encrypted: " + enc);
        System.out.println("Decrypted: " + cipher(enc, -1));
    }
}
```

3. HillCipher

```
import java.util.Scanner;

class HillCipher {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter text: ");
        String text = sc.nextLine().toUpperCase().replace(" ", "");

        if (text.length() % 2 != 0)
            text += "X";

        String enc = "", dec = "";

        for (int i = 0; i < text.length(); i += 2) {
            int a = text.charAt(i) - 'A';
            int b = text.charAt(i + 1) - 'A';

            enc += (char) ((3 * a + 3 * b) % 26 + 'A');
            enc += (char) ((2 * a + 5 * b) % 26 + 'A');
        }

        for (int i = 0; i < enc.length(); i += 2) {
            int a = enc.charAt(i) - 'A';
            int b = enc.charAt(i + 1) - 'A';

            dec += (char) ((15 * a + 17 * b) % 26 + 'A');
            dec += (char) ((20 * a + 9 * b) % 26 + 'A');
        }

        System.out.println("Encrypted: " + enc);
        System.out.println("Decrypted: " + dec);
```

```
        sc.close();
    }
}
```

4. RailFence

```
import java.util.Scanner;

class RailFence {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter text: ");
        String text = sc.nextLine().replace(" ", "");

        System.out.print("Enter number of rails: ");
        int key = sc.nextInt();

        char[][] rail = new char[key][text.length()];

        int row = 0, dir = 1;

        for (int i = 0; i < text.length(); i++) {
            rail[row][i] = text.charAt(i);

            if (row == 0) dir = 1;
            else if (row == key - 1) dir = -1;

            row += dir;
        }

        String enc = "";

        for (int i = 0; i < key; i++)
            for (int j = 0; j < text.length(); j++)
                if (rail[i][j] != 0)
                    enc += rail[i][j];

        System.out.println("Encrypted: " + enc);

        String dec = "";
        row = 0;
        dir = 1;
```

```
        for (int i = 0; i < text.length(); i++) {
            dec += rail[row][i];

            if (row == 0) dir = 1;
            else if (row == key - 1) dir = -1;

            row += dir;
        }

        System.out.println("Decrypted: " + dec);
        sc.close();
    }
}
```

5. Row column transformations

```
import java.util.Scanner;

class RowColumn {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter text: ");
        String s = sc.nextLine().replace(" ", "");

        System.out.print("Enter columns: ");
        int k = sc.nextInt();

        while (s.length() % k != 0)
            s += "X";

        int r = s.length() / k;
        String enc = "", dec = "";

        for (int j = 0; j < k; j++)
            for (int i = j; i < s.length(); i += k)
                enc += s.charAt(i);

        System.out.println("Encrypted: " + enc);

        for (int i = 0; i < r; i++)
            for (int j = 0; j < k; j++)
                dec += enc.charAt(j * r + i);

        System.out.println("Decrypted: " + dec);
        sc.close();
```

```
    }
}
```

6. DES-python

Bash : pip install pycryptodome

```
from Crypto.Cipher import DES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

key = get_random_bytes(8)
cipher = DES.new(key, DES.MODE_ECB)

text = input("Enter text: ").encode()

enc = cipher.encrypt(pad(text, 8))
print("Encrypted:", enc.hex())

dec = DES.new(key, DES.MODE_ECB)
plain = unpad(dec.decrypt(enc), 8)
print("Decrypted:", plain.decode())
```

7.AES-python

Bash : pip install pycryptodome

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

key = get_random_bytes(16)
cipher = AES.new(key, AES.MODE_ECB)

text = input("Enter text: ").encode()

enc = cipher.encrypt(pad(text, 16))
print("Encrypted:", enc.hex())

cipher = AES.new(key, AES.MODE_ECB)
dec = unpad(cipher.decrypt(enc), 16)
print("Decrypted:", dec.decode())
```

8. Electronic Codebook (ECB) -python

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

key = b'1234567890123456'
text = input("Enter plaintext: ").encode()

cipher = AES.new(key, AES.MODE_ECB)
encrypted = cipher.encrypt(pad(text, 16))
print("Encrypted:", encrypted.hex())

cipher = AES.new(key, AES.MODE_ECB)
decrypted = unpad(cipher.decrypt(encrypted), 16)
print("Decrypted:", decrypted.decode())
```

9.Cipher Block Chaining (CBC) -python

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

key = b'1234567890123456'
iv = b'abcdefghijklmnop'

text = input("Enter plaintext: ").encode()

cipher = AES.new(key, AES.MODE_CBC, iv)
encrypted = cipher.encrypt(pad(text, 16))
print("Encrypted:", encrypted.hex())

cipher = AES.new(key, AES.MODE_CBC, iv)
decrypted = unpad(cipher.decrypt(encrypted), 16)
print("Decrypted:", decrypted.decode())
```

10.RC4 -python

```
def rc4(text, key):
    S = list(range(256))
    j = 0

    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
```

```
    i = j = 0
    result = ""

    for ch in text:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        k = S[(S[i] + S[j]) % 256]
        result += chr(ord(ch) ^ k)

    return result

text = input("Enter plaintext: ")
key = input("Enter key: ")

encrypted = rc4(text, [ord(c) for c in key])
print("Encrypted:", encrypted.encode().hex())

decrypted = rc4(encrypted, [ord(c) for c in key])
print("Decrypted:", decrypted)
```

11.RSA -python

```
p = 3
q = 11

n = p * q
phi = (p - 1) * (q - 1)

e = 3
d = 7

msg = int(input("Enter a number to encrypt (less than 33): "))

encrypted = pow(msg, e, n)
decrypted = pow(encrypted, d, n)

print("Encrypted message:", encrypted)
print("Decrypted message:", decrypted)
```

12.Diffie-Hellman Key Exchange

```
p = 23
g = 5

a = int(input("Enter private key of Alice: "))
b = int(input("Enter private key of Bob: "))

A = pow(g, a, p)
B = pow(g, b, p)

print("Alice's public key:", A)
print("Bob's public key:", B)

key1 = pow(B, a, p)
key2 = pow(A, b, p)

print("Alice's shared key:", key1)
print("Bob's shared key:", key2)
```

13.Message digest

```
import hashlib

text = input("Enter text: ")

hash_value = hashlib.sha512(text.encode()).hexdigest()

print("SHA-512 Digest:", hash_value)
```

14. Digital Signature

```
import java.security.*;
import java.util.*;

class DigitalSignature {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter text: ");
        String text = sc.nextLine();

        KeyPairGenerator kpg = KeyPairGenerator.getInstance("RSA");
        kpg.initialize(2048);
        KeyPair kp = kpg.generateKeyPair();
```

```
        Signature s = Signature.getInstance("SHA256withRSA");
        s.initSign(kp.getPrivate());
        s.update(text.getBytes());
        byte[] sig = s.sign();

        System.out.println("Signature: " + Base64.getEncoder().encodeToString(sig));

        s.initVerify(kp.getPublic());
        s.update(text.getBytes());
        System.out.println("Verified: " + s.verify(sig));

        sc.close();
    }
}
```

1. Firewall: Features and setting Firewall Security in Windows

Definition: A firewall is a security system that monitors and controls incoming and outgoing network traffic based on predefined rules.

Features of a firewall:

1. Inbound and outbound rules: Controls incoming and outgoing network traffic.
2. Blocks unauthorized access: Prevents unwanted network connections.
3. Program exceptions: Allows trusted programs to communicate through the network.
4. Network profiles: Supports Public, Private and Domain profiles.
5. Stealth feature: Helps hide the computer from certain network probing techniques.
6. Boot-time filtering: Starts filtering network traffic when the network interface becomes active.

Steps to enable Firewall in Windows:

1. Open Control Panel.
2. Select System and Security.
3. Click Windows Defender Firewall.
4. Select Turn Windows Defender Firewall on or off.
5. Enable it for Private and Public network settings.
6. Click OK.

Conclusion: A firewall protects a computer by blocking unauthorized network traffic and allowing trusted connections.

2. Steps to ensure security of a web browser (Google Chrome)

Definition: Browser security protects users from malicious websites, unsafe downloads, tracking and online attacks.

Security steps:

1. Keep Chrome updated: Install the latest security updates.
2. Enable Safe Browsing: Open Settings → Privacy and security → Security and choose an appropriate Safe Browsing level.
3. Use HTTPS: Prefer websites that use secure HTTPS connections.
4. Block pop-ups: Open Settings → Privacy and security → Site settings → Pop-ups and redirects.
5. Manage cookies: Block third-party cookies where appropriate.
6. Check extensions: Remove unknown or unnecessary extensions.
7. Avoid saving passwords on shared devices: Use a trusted password manager.
8. Avoid suspicious links and downloads: Download files only from trusted websites.
9. Use strong passwords and two-factor authentication for online accounts.
10. Clear browsing data: Remove cookies, cache and history when necessary.

Conclusion: These steps help protect personal information and reduce browser security risks.

3. Types of vulnerabilities in websites / web applications

Definition: A web application vulnerability is a weakness in a website that attackers can exploit to access data or perform unauthorized actions.

Important types of vulnerabilities:

1. SQL Injection: Attackers insert malicious SQL queries to access or modify database information.
2. Cross-Site Scripting (XSS): Attackers inject malicious scripts that execute in a user's browser.
3. Broken Authentication: Weak login or authentication mechanisms allow unauthorized access to user accounts.
4. Cross-Site Request Forgery (CSRF): Tricks an authenticated user into performing unwanted actions.
5. Security Misconfiguration: Incorrect security settings, default accounts or unpatched software create weaknesses.
6. Sensitive Data Exposure: Confidential information, such as passwords and financial details, is not properly protected.
7. Broken Access Control: Users gain access to resources or functions they are not authorized to use.

8. Known Vulnerabilities: Attackers exploit weaknesses in outdated software or components.
9. Denial of Service (DoS): Floods a service with requests, making it unavailable to legitimate users.
10. Unvalidated Redirects: Redirect users to malicious or phishing websites.

Conclusion: These vulnerabilities can be reduced through input validation, secure authentication, access control, software updates and security testing.

4. Analysis of security vulnerabilities in email applications

Definition: Email vulnerabilities are weaknesses that attackers exploit to read messages, steal information, impersonate users or spread malware.

Important threats in email communication:

1. Eavesdropping: Unauthorized persons read emails during transmission.
2. Identity Theft: Attackers steal login details and impersonate users.
3. Message Modification: Email content is altered without permission.
4. Email Spoofing: Attackers forge the sender's email address.
5. Spam: Unwanted or bulk emails are sent to users.
6. Email Bombing: A large number of emails are sent to overwhelm an inbox or system.
7. Phishing: Fake emails trick users into revealing passwords or banking details.
8. Malware Attachments: Infected files spread viruses or other malicious software.
9. Email Fraud: Emails deceive users for financial or personal gain.
10. Unprotected Backups: Stored emails and backups may expose confidential information.

Security services required for email:

1. Confidentiality: Prevents unauthorized persons from reading messages.
2. Integrity: Ensures messages are not modified.
3. Message Authentication: Verifies the sender's identity.
4. Non-repudiation: Helps prevent the sender from denying a message they sent.
5. Entity Authentication: Verifies users before allowing access.

Limitations of email protocols:

- SMTP: Without encryption, messages and login details may be exposed.
- POP and IMAP: Login credentials may be exposed if secure connections are not used.

Precautions: Use strong passwords, multi-factor authentication, secure email connections, spam filters and avoid suspicious links or attachments.

Conclusion: Email security can be improved by encryption, authentication and safe email practices.

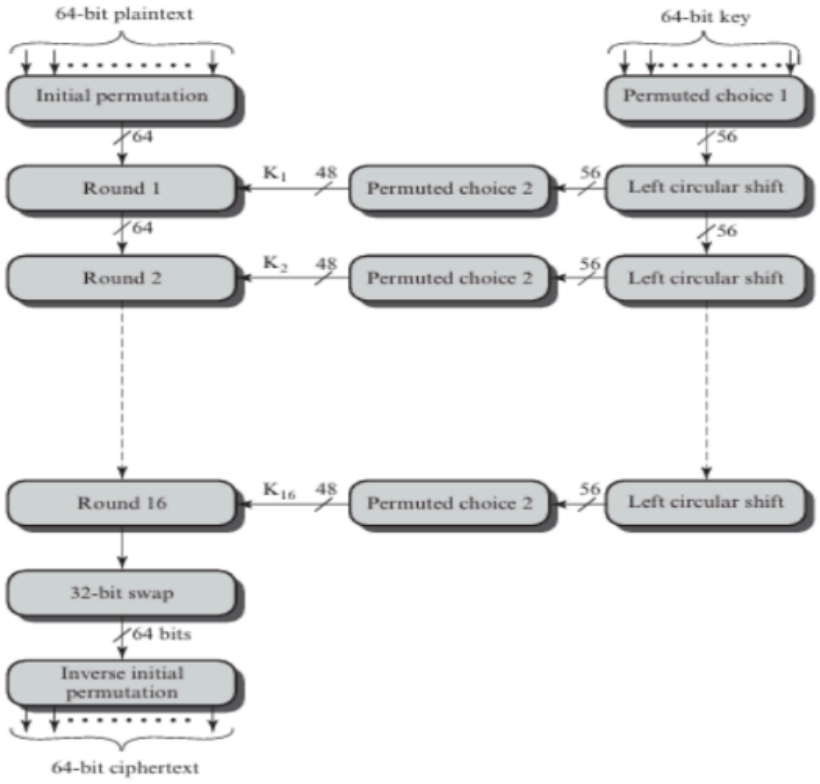


Figure 4.5 General Depiction of DES Encryption Algorithm

- **DES is a Feistel cipher with the exception of IP and IP^{-1}.**

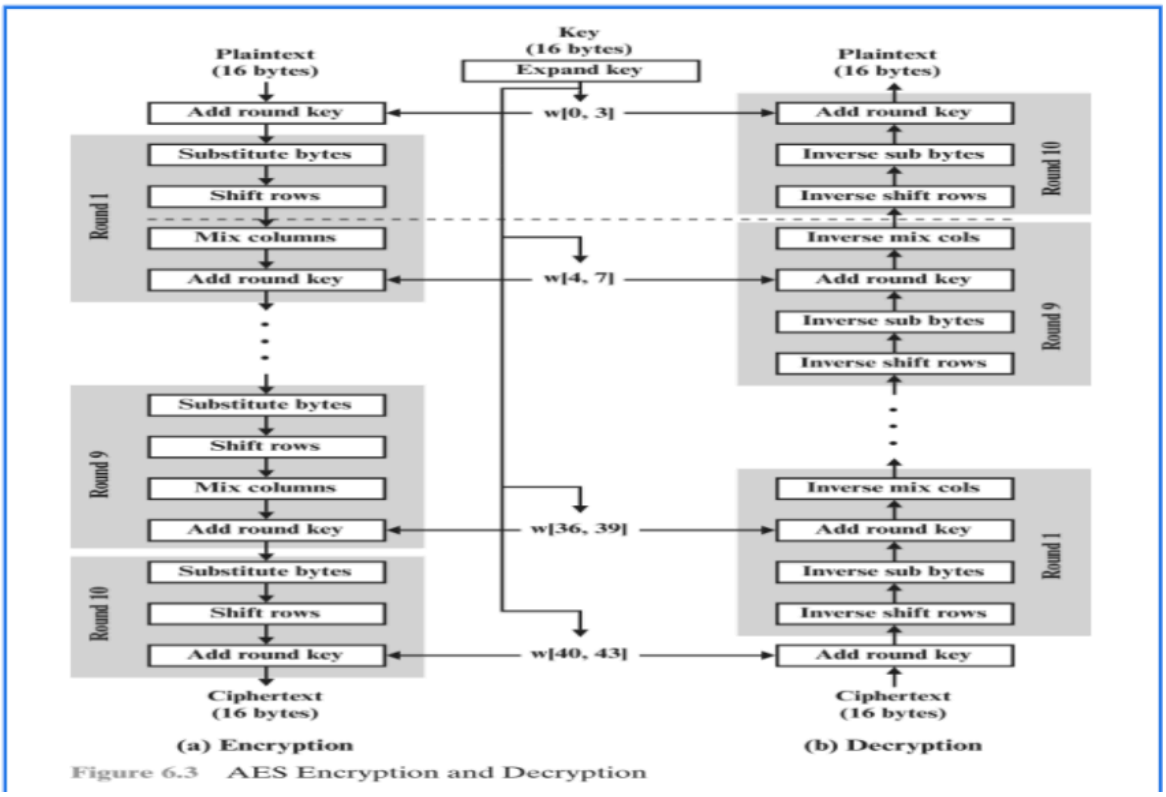
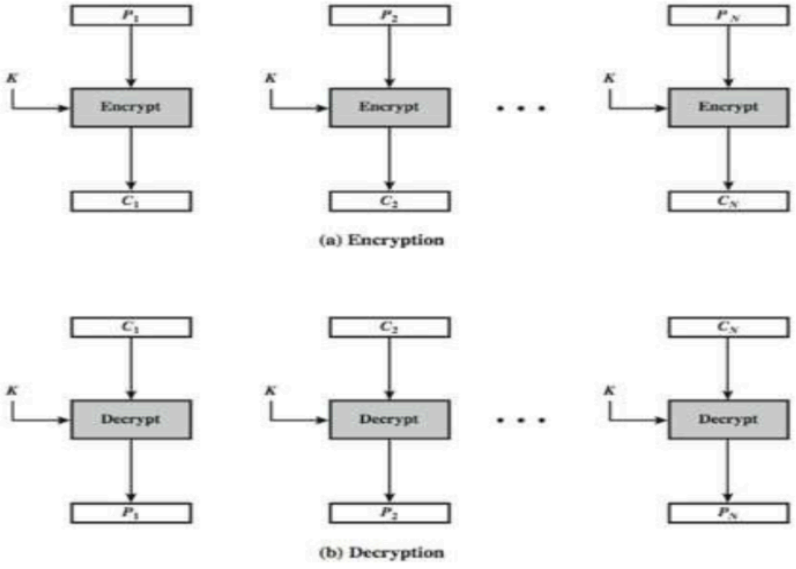
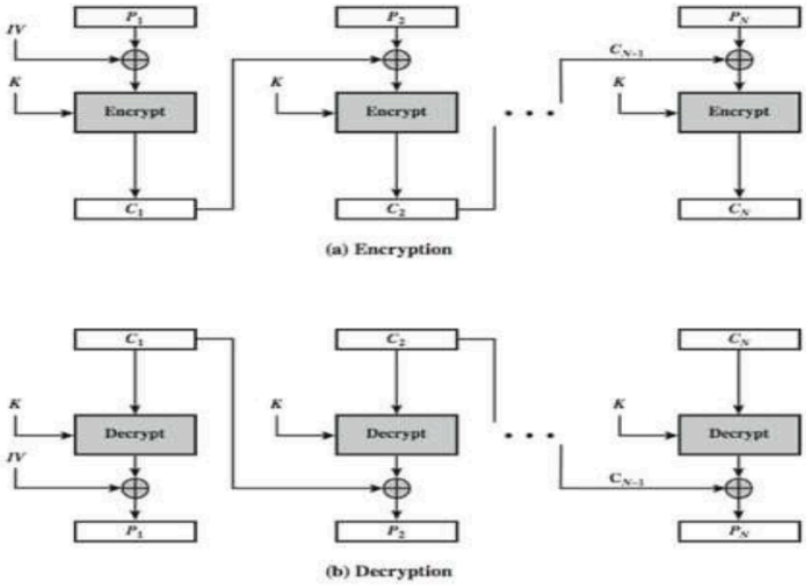


Figure 6.3 AES Encryption and Decryption

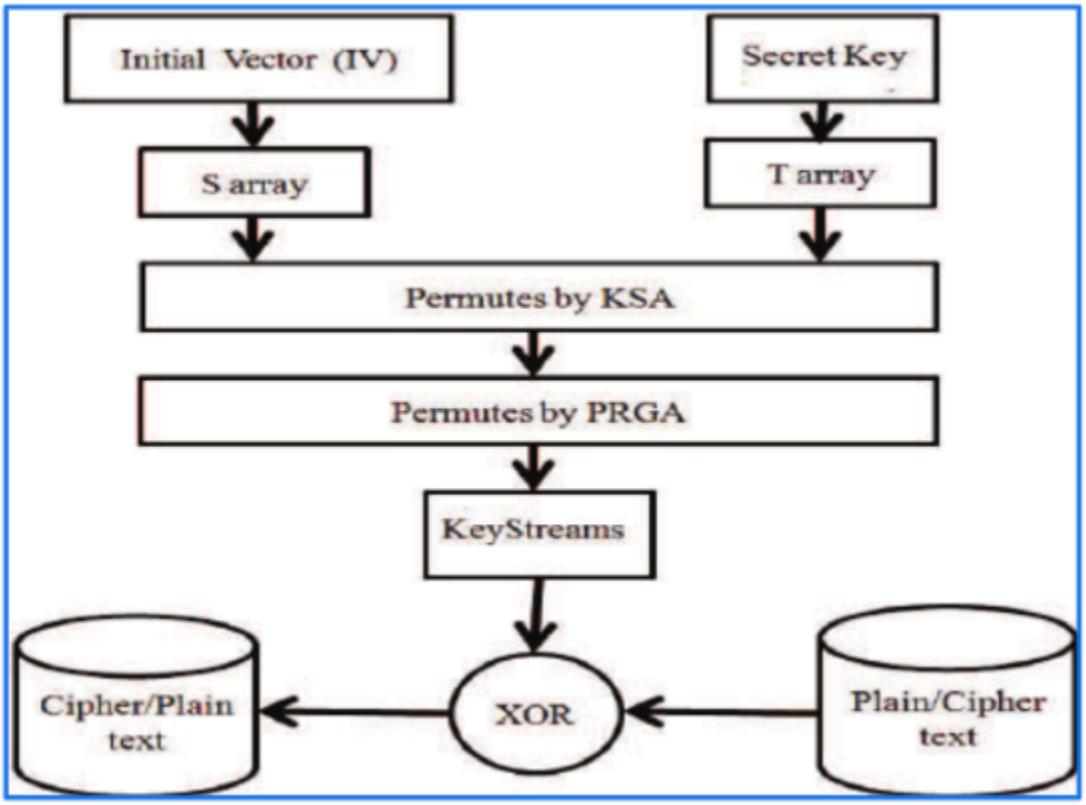
Electronic Codebook Book (ECB



Cipher Block Chaining (CBC)



RC4



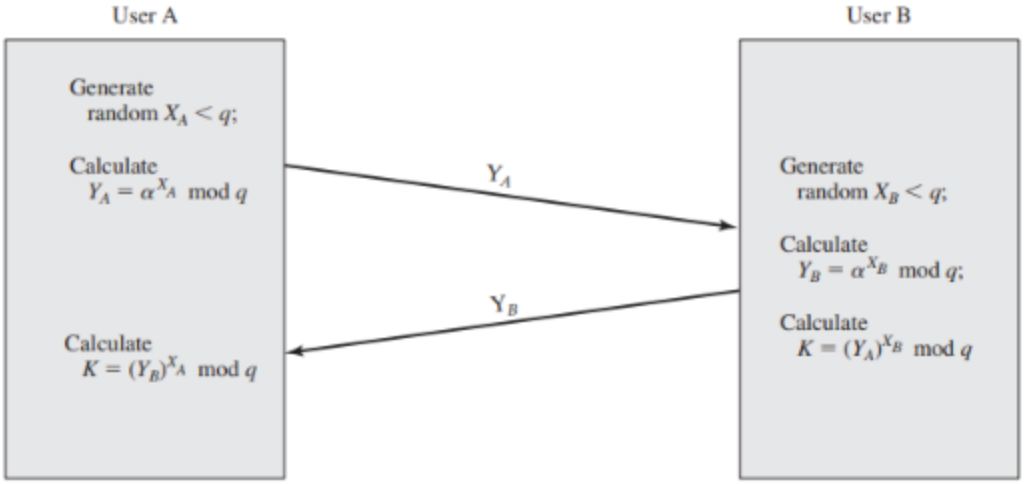


Figure 3.13 Diffie-Hellman Key Exchange

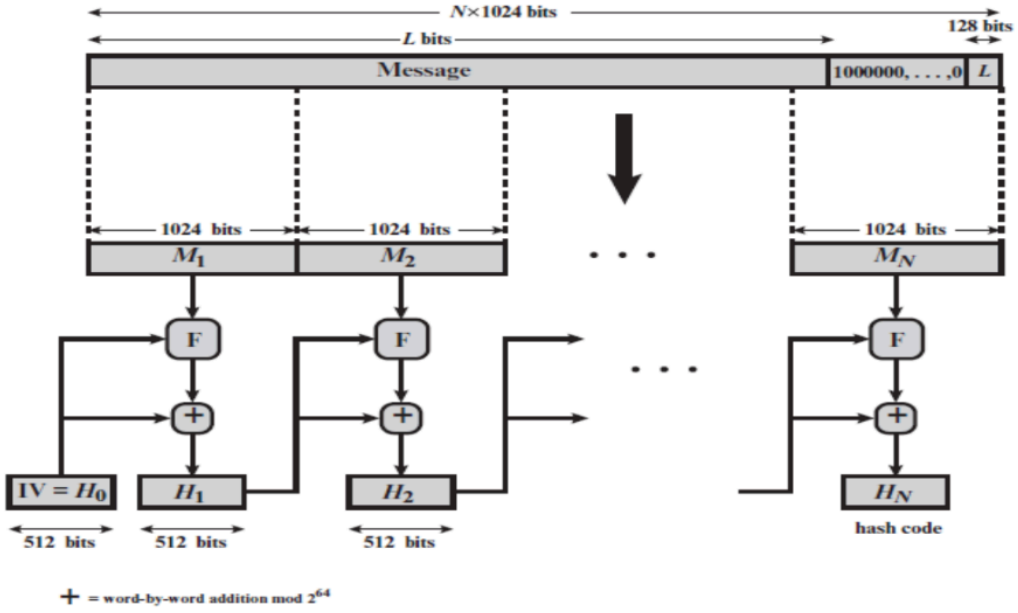


Figure 11.9 Message Digest Generation Using SHA-512

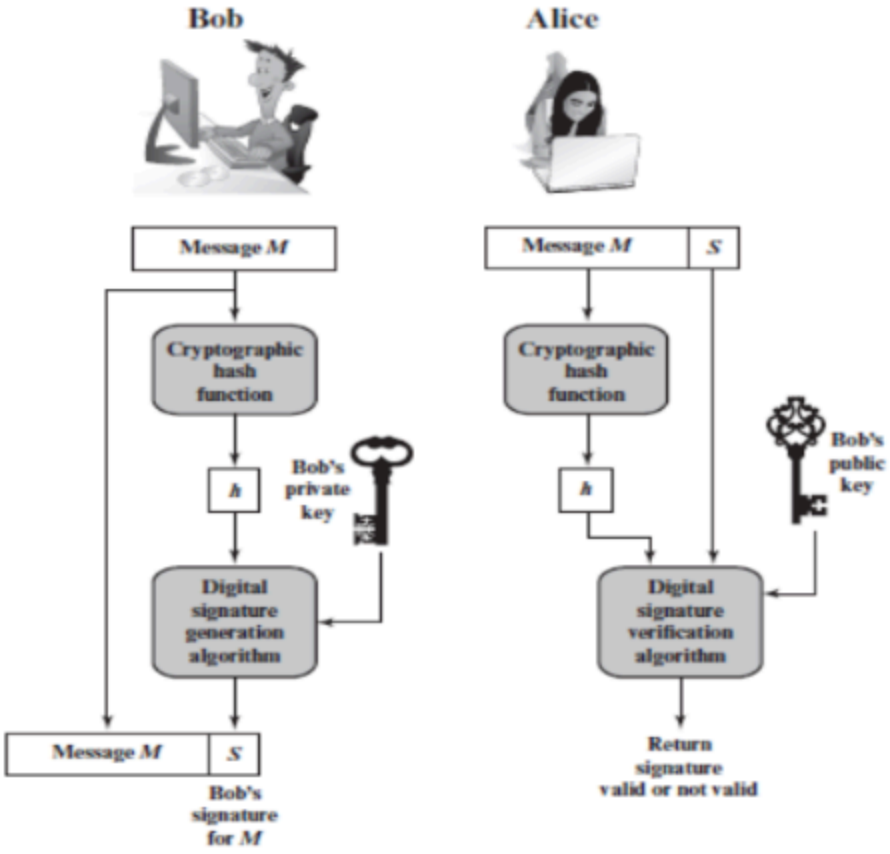


Figure 13.1 Simplified Depiction of Essential Elements of Digital Signature Process